



Software Engineering Department

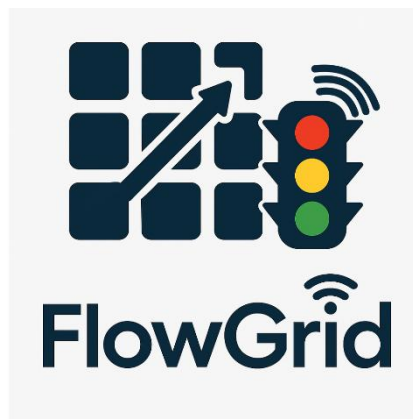
Braude College of Engineering

Final Project – Phase A (61998)

Adaptive Traffic Light Control System

**Real-Time Intersection Optimization Empowered by Computer Vision and Deep
Reinforcement Learning**

CODE: 26-1-D-30



By:

Avishag Levi

Einav Momi Ben Shushan

Advisor:

Dr. Cohen Reuven

Link to GitHub:

<https://github.com/einavbs1/FlowGrid>

Abstract

Traffic congestion is a rapidly growing global challenge, leading to significant economic losses, environmental damage, and delayed emergency response times. Traditional traffic management systems, such as Fixed-Time controllers and inductive loop sensors, fail to adapt to real-time traffic dynamics, resulting in inefficient "green time" allocation and increased waiting times.

This project, the Adaptive Traffic Light Control System, introduces a software-centric solution designed to retrofit existing intersections using a "Camera-as-a-Sensor" approach. The system leverages advanced Computer Vision (YOLO) for real-time vehicle detection and classification, Object Tracking (DeepSORT) for precise queue metric calculation, and Deep Reinforcement Learning (DQN) for autonomous decision-making.

The core objective is to minimize cumulative waiting times and prioritize critical services. The system features a Priority Protocol that ensures zero delay for emergency vehicles and improves public transit efficiency. To guarantee safety and reliability, a Rule-Based Safety Layer and a Historical Data Fallback Mode are integrated to mitigate AI "black box" risks and sensor failures.

Evaluation conducted within the SUMO simulation environment demonstrates the system's superiority over standard controllers, targeting a 20% reduction in average waiting time and ensuring immediate preemption for emergency vehicles. This project proves that intelligent, adaptive control can be achieved cost-effectively using existing infrastructure, bridging the gap between legacy systems and future V2X technologies.

Keywords: Intelligent Transportation Systems (ITS), Deep Reinforcement Learning (DQN), Computer Vision, YOLO, SUMO Simulation, Traffic Signal Control.

Table of Contents

Abstract	2
1. Introduction	6
1.1. Background and Problem Statement	6
1.2. Existing Solutions and Their Limitations	6
1.3. The Proposed Solution	6
1.4. Stakeholders	6
1.5. Document Structure	7
2. Background and Related Work	7
2.1. Existing Solutions and Approaches	7
2.1.1. Fixed Time (Pre timed) Control	7
2.1.2. Vehicle Actuated Control and Inductive Loop Detection	8
2.1.3. Adaptive Network Control (SCOOT)	8
2.1.4. Adaptive Logic and Degree of Saturation (SCATS)	8
2.2. Future Research Directions	8
2.2.1. Integration of V2X with Deep Reinforcement Learning	8
2.2.2. Autonomous Intersection Management (AIM)	8
2.3. Algorithmic Foundations	8
2.3.1. Visual Sensing and The YOLO Architecture	9
2.3.2. Object Tracking and Association (DeepSORT)	9
2.3.3. Deep Reinforcement Learning (DQN)	9
2.3.4. Recent Approaches (Sim-to-Real)	9
2.4. Summary and Research Gap	9
3. Requirement Gathering and Analysis Process	9
3.1. Market Research and Paradigm Analysis	10
3.2. Literature Review and Technical Constraints	10
3.3. Safety and Failure Analysis	10
3.4. Stakeholder Needs	10
4. Engineering Process	11
4.1. Research Phase & Strategy	11
4.2. Planned Interface Strategy (Co-Simulation)	11
4.3. Deep Reinforcement Learning (DQN) Architecture	12
4.3.1. Proposed State Space (St)	12
4.3.2. Action Space Definition (At)	12
4.3.3. Reward Function Design (Rt)	12

4.3.4.	Network Architecture Plan We plan to implement a Multi-Layer Perceptron (MLP) architecture:.....	13
4.4.	Data Sources and Training Strategy	13
4.5.	Constraints & Challenges	14
4.6.	Development Stages	14
5.	Expected Achievements.....	14
5.1.	Primary Objectives	14
5.2.	Expected System Capabilities.....	15
6.	Tools and Technologies.....	15
6.1.	Computer Vision (YOLOv8)	15
6.2.	Object Tracking (DeepSORT)	15
6.3.	Simulation & Development (SUMO / Python)	16
6.4.	Web & Cloud Infrastructure	16
6.5.	Development Libraries & Hardware.....	16
7.	Success Criteria.....	16
.8	Project Requirements	17
8.1.	Functional Requirements (FR).....	17
8.2.	Non-Functional Requirements (NFR).....	18
9.	Verification and Evaluation	19
9.1.	Test Plan Overview	19
9.2.	Defined Test Scenarios	19
9.3.	Test Environment	19
9.4.	Evaluation Methodology:	19
9.5.	Key Performance Metrics.....	20
9.6.	Verification Test Cases	20
9.7.	Evaluation Metrics.....	21
.10	User Interface Design	22
10.1.	Overview & Layout.....	22
10.2.	UI Components	22
10.3.	System Behavior.....	22
11.	Use Case Diagram Description	23
11.1.	Overview	23
11.2.	Actors	23
11.3.	Functional Capabilities.....	23
11.4.	Dependencies The diagram utilizes an <<include>> relationship:	24
12.	Activity Diagram Description –.....	24

12.1.	Overview	24
12.2.	Structure (Swimlanes)	24
12.3.	Process Flow.....	24
13.	Deployment Diagram Description –	25
13.1.	Overview	25
13.2.	Nodes & Execution Environments	25
13.3.	Communication Protocols.....	26
14.	References	27
.15	Appendices.....	28
15.1.	Appendix A: State Space Representation Pipeline.....	28
15.2.	Appendix B: Use of Generative AI	28
15.2.1.	AI Tools Utilized	28
15.2.2.	Prompts	28

1. Introduction

1.1. Background and Problem Statement

Traffic congestion is a critical urban challenge caused by infrastructure that fails to adapt to dynamic vehicle volumes. The core inefficiency lies in static, fixed-time traffic lights and basic reactive sensors (induction loops). These systems are "blind" to real-time parameters like queue length, wasting green time on empty lanes while congestion builds elsewhere. This results in significant economic losses, increased CO2 emissions, and safety risks.

1.2. Existing Solutions and Their Limitations

Current management approaches generally fall into two extremes: Actuated Systems, which rely on binary induction loops that optimize locally but lack traffic density quantification and Connected Vehicle Networks (V2X), which are theoretically optimal but currently impractical due to high infrastructure costs and low market penetration. Our project bridges this gap by utilizing a "Camera-as-a-Sensor" approach, offering intelligent control using accessible technology without waiting for future standardization.

1.3. The Proposed Solution

We propose an Adaptive Traffic Light Control System that leverages Computer Vision and AI to make real-time decisions. The system architecture integrates three core domains:

- **Computer Vision (YOLO):** Performs real-time detection and classification to distinguish standard traffic from high-priority vehicles (ambulances, buses).
 - **Object Tracking (DeepSORT):** Assigns unique IDs to vehicles to calculate precise waiting times and true queue lengths.
 - **Deep Reinforcement Learning (DQN):** An agent that learns to minimize waiting times based on visual data, featuring a Priority Protocol for emergency preemption.
- ❖ To ensure reliability, the system includes a Historical Data Fallback Mode and a safety layer, validated via the SUMO simulation platform.

1.4. Stakeholders

The system maximizes utility for the entire urban ecosystem:

- **Emergency Services:** Drastically reduced response times via immediate signal preemption.
- **Public Transportation:** Improved efficiency through green light priority, incentivizing public transit usage.
- **Daily Commuters:** Reduced travel times and smoother traffic flow.

- **Municipalities:** A cost effective solution to upgrade intersections without expensive physical infrastructure projects.
- **The Environment:** Lower CO2 emissions and fuel consumption due to reduced stop and go cycles.

1.5. Document Structure

The remainder of this document is organized as follows:

- **Background and Related Work:** Reviews existing traffic paradigms and algorithmic foundations (YOLO, DQN).
- **Requirement Gathering:** Details market research, safety analysis, and stakeholder needs.
- **Engineering Process:** Explains the research strategy, simulation interface, and DQN architecture.
- **Expected Achievements:** Outlines, primary objectives and expected system capabilities.
- **Tools and Technologies:** Lists the technical stack, including YOLOv8, DeepSORT, and SUMO.
- **Success Criteria:** Defines the quantitative standards for project success.
- **Project Requirements:** Specifies the complete Functional and Non-Functional requirements.
- **Verification and Evaluation:** Describes the test plan, scenarios, and evaluation methodology.
- **System Design Diagrams:** Presents the UI, Use Case, Activity, and Deployment descriptions.
- **References & Appendices:** Lists citations and supplementary materials (GenAI report, extra diagrams).

2. Background and Related Work

2.1. Existing Solutions and Approaches

Traffic signal control has evolved from basic schedules to complex adaptive systems. Current implementations generally fall into four levels of sophistication:

2.1.1. Fixed Time (Pre timed) Control

Operates on rigid schedules derived from historical data. According to the Traffic Signal Timing Manual, while this method offers predictability, it lacks flexibility and operates "blindly" to real-time variations, often wasting green time on empty lanes.

2.1.2. Vehicle Actuated Control and Inductive Loop Detection

Utilizes Inductive Loop Detectors to respond to real-time demand. However, as noted by Akcelik (1994), it relies on binary detection (presence of metal) rather than volume analysis and entails high installation and maintenance costs due to the need for physical road excavation. Consequently, a single vehicle on a side street can inefficiently interrupt heavy flow on a main artery.

2.1.3. Adaptive Network Control (SCOOT)

A network-wide architecture that utilizes upstream detectors to model traffic platoons. It eliminates pre-calculated plans and continuously optimizes signal timings (Split, Cycle, Offset) to maintain a "Green Wave" based on actual conditions.

2.1.4. Adaptive Logic and Degree of Saturation (SCATS)

Operates on an "Equisaturation" philosophy using stop-line detectors. It measures the Degree of Saturation (DS) and reallocates green time to equalize the load across conflicting approaches, maintaining optimal flow density.

2.2. Future Research Directions

V2X and Autonomous Control Academic research has shifted towards rethinking intersection management entirely, envisioning a future where vehicle to infrastructure communication replaces physical signals.

2.2.1. Integration of V2X with Deep Reinforcement Learning

Liang et al. (2018) demonstrated the use of V2I communication to create high-resolution grid states for a Double Dueling Deep Q-Network (D3QN) agent. While this approach significantly reduces waiting times, its reliance on high market penetration of connected vehicles makes it currently impractical for widespread deployment.

2.2.2. Autonomous Intersection Management (AIM)

Dresner and Stone (2008) proposed a theoretical, reservation-based protocol that removes traditional Red/Green phases. Vehicles send trajectory requests to an Intersection Manager, allowing them to "weave" through intersections simultaneously, creating a granular flow that drastically outperforms traffic signals.

2.3. Algorithmic Foundations

Computer Vision and Object Detection This section reviews the core algorithms required to implement a practical vision based system, focusing on Real Time Object Detection and its application in traffic control.

2.3.1. Visual Sensing and The YOLO Architecture

Proposed by Redmon et al. (2016), YOLO utilizes a unified CNN architecture to frame detection as a single regression problem. Unlike slower multi-stage systems (e.g., R-CNN), it divides the image into a grid to predict bounding boxes simultaneously. This enables high frame rates (45+ FPS), providing real-time data on queue lengths and classification without computational bottlenecks.

2.3.2. Object Tracking and Association (DeepSORT)

To address YOLO's lack of temporal memory, DeepSORT combines a recursive Kalman Filter (for position prediction) with a CNN (for appearance vectors) using the Hungarian Algorithm. This ensures vehicles retain unique IDs and waiting time counters even during occlusion, enabling precise metric calculation.

2.3.3. Deep Reinforcement Learning (DQN)

Based on Mnih et al. (2015), the system models traffic control as a Markov Decision Process (MDP). The "State" is the visual snapshot, the "Action" is the phase change, and the "Reward" is the negative cumulative waiting time. The Agent approximates the Q-function to learn optimal policies through trial and error.

2.3.4. Recent Approaches (Sim-to-Real)

Wang et al. (2024) proposed a "Traffic Co-Simulation Framework" that utilizes real-time video feeds to drive DRL agents. This research validates that camera-based autonomous control can effectively handle non-linear traffic patterns that traditional sensors fail to capture.

2.4. Summary and Research Gap

The literature reveals a distinct gap between deployed legacy systems (SCATS), which lack true intelligence, and theoretical future models (V2X), which are currently impractical due to infrastructure costs. This project bridges this gap by offering an immediate, deployable solution that leverages standard "Camera-as-a-Sensor" technology (YOLO/RL) to provide intelligent control today, without waiting for a fully autonomous vehicle fleet.

3. Requirement Gathering and Analysis Process

The system requirements were derived from a structured engineering research process comprising four primary analysis phases. This process transformed abstract problems into concrete technical constraints, ensuring traceability between the research findings and the final system specifications (presented in Chapter 8).

3.1. Market Research and Paradigm Analysis

Our comparative analysis of legacy systems focused on their optimization logic. We identified that systems like SCATS prioritize "Intersection Capacity" (total cars passed) rather than individual driver experience. This gap directly drove the decision to define "Cumulative Waiting Time Minimization" as our primary optimization function. Furthermore, the economic analysis of V2X revealed that its dependency on high market penetration makes it irrelevant for current infrastructure. This constraint led to the architectural decision to adopt a "Software-Retrofit" approach (formalized as NFR-01 in Chapter 8), requiring the system to function using standard, existing camera hardware.

3.2. Literature Review and Technical Constraints

the "Camera-as-a-Sensor" concept into engineering requirements revealed distinct hardware limitations. Analyzing the velocity of vehicles entering an intersection indicated that preventing motion blur and ensuring high-confidence detection requires high-frequency processing. This physical constraint dictated the non-functional requirement for a minimum processing rate of 45 FPS (see NFR-13 in Chapter 8), effectively ruling out slower, multi-stage detection architectures. Additionally, the need to calculate "True Waiting Time" highlighted a gap in standard object detection—specifically the lack of "Object Permanence." This necessitated the integration of a specific tracking layer (DeepSORT) capable of maintaining vehicle identities across temporal frames.

3.3. Safety and Failure Analysis

Since Deep Reinforcement Learning models are inherently non-deterministic "black boxes," relying solely on the AI agent poses a safety risk. To address this, we conducted a failure mode analysis which determined that a Deterministic Override mechanism is mandatory. This resulted in the definition of the Rule-Based Safety Layer (defined as FR-12 in the Project Requirements), which strictly enforces minimum green times regardless of the AI's output. Simultaneously, we addressed the risk of sensor blindness (due to weather or occlusion) by defining a Historical Data Fallback Mode (FR-14), ensuring the intersection never freezes even if the vision module fails.

3.4. Stakeholder Needs

Interviews regarding emergency response protocols highlighted an operational conflict: standard optimization algorithms treat all vehicles equally. However, from a municipal and safety perspective, an ambulance has critical priority compared to a private car. Resolving this conflict required a deviation from the standard reward function, leading to the Signal Preemption requirements (FR-09, FR-10; detailed in

Chapter 8). This feature forces the system to abandon its optimization logic temporarily to provide an immediate "Green Wave" for identified emergency vehicles.

4. Engineering Process

This section outlines the development lifecycle of our Adaptive Traffic Light Control System, detailing the research phases, methodological choices such as the shift to computer vision and the engineering constraints that shaped the final architecture.

4.1. Research Phase & Strategy

Our initial investigation into V2X technology confirmed its current impracticality due to low market penetration. Consequently, we adopted a strategy focused on retrofitting existing infrastructure via computer vision. Supported by Wang et al. (2024), we identified that relying solely on static datasets leads to poor generalization. Therefore, we adopted a "Simulation-First" approach.

- **Strategic Tool Selection: SUMO** To facilitate this strategy, we selected SUMO (Simulation of Urban MObility) as our primary testbed. Unlike standard microscopic simulators, SUMO offers the TraCI (Traffic Control Interface) API, which was critical for our engineering process. It allowed us to decouple the control logic (Python/DQN) from the physical environment, enabling us to train the AI agent on millions of synthetic traffic scenarios safely before considering any real-world deployment. This validated our "Camera-as-a-Sensor" concept without the cost and risk of physical field testing.

4.2. Planned Interface Strategy (Co-Simulation)

To bridge the gap between the simulation environment and the control logic, we plan to utilize the TraCI (Traffic Control Interface). This architecture is designed to function as a "Co-Simulation" loop, allowing for external processing of visual data and real-time command execution. By decoupling the logic from the physics engine, this approach will enable the Python-based agent to retrieve synthetic visual data and execute traffic light commands instantly. This closes the feedback loop, ensuring that every action taken by the model will be immediately reflected in the simulation state.

RL Interaction Loop (Training Process) Co-Simulation between Python and SUMO

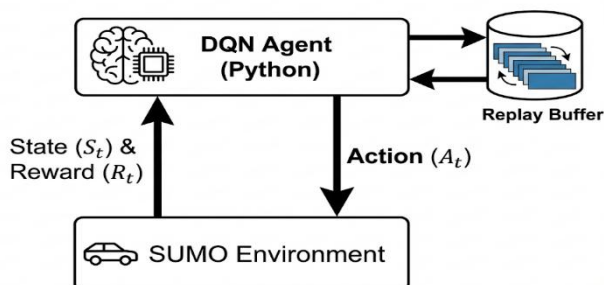


Figure 1: Agent-Environment Interaction Loop – visualizing the cyclical data flow (State, Action, Reward) between the Python Control Logic and the SUMO Simulation via TraCI.

4.3. Deep Reinforcement Learning (DQN) Architecture

The core decision-making engine of the system is designed to be a Deep Q-Network (DQN) agent. We selected this architecture because, unlike traditional heuristic algorithms (which rely on fixed rules), a DQN agent can learn optimal control policies (π^*) by interacting directly with the environment. This capability will allow the system to adapt to non-linear and unpredictable traffic patterns that standard sensors fail to optimize. We propose to model the problem as a Markov Decision Process (MDP) defined by the tuple (S, A, R, γ) :

4.3.1. Proposed State Space (S_t)

To enable the agent to "see" the intersection, we will convert the raw visual data into a structured numerical tensor (illustrated in [Appendix A, Figure 3](#)). The state at time t will be defined as a discrete grid representation:

$$S_t = [P_N, P_S, P_E, P_W, L_{phase}] \text{ Where:}$$

- P_i : Will represent the Queue Density matrix for each arm (North, South, East, West), derived from the YOLO detection output.
- L_{phase} : Will be a One-Hot Encoded vector representing the currently active green phase.

This design is intended to ensure that the neural network receives both the traffic volume and the current signal context.

4.3.2. Action Space Definition (A_t)

The agent will operate in a discrete action space designed to maintain safety constraints. At every step (e.g., every 5 seconds), the agent will choose one of two actions:

1. Keep Phase ($a=0$): Extend the current green light duration.
2. Switch Phase ($a=1$): Initiate the transition sequence to the next conflicting phase.

4.3.3. Reward Function Design (R_t)

The reward function will serve as the critical component guiding the learning process. Our objective is to minimize total waiting time. Therefore, we plan to define the reward as the negative cumulative waiting time of all vehicles:

$$R_t = - \left(\sum_{i=1}^N w_{i,t} + \alpha \cdot Q_{spillover} \right)$$

Advantages of this approach: By maximizing this negative value (bringing it closer to zero), the agent is expected to learn implicitly how to clear queues as fast as possible, prioritizing lanes with higher delays.

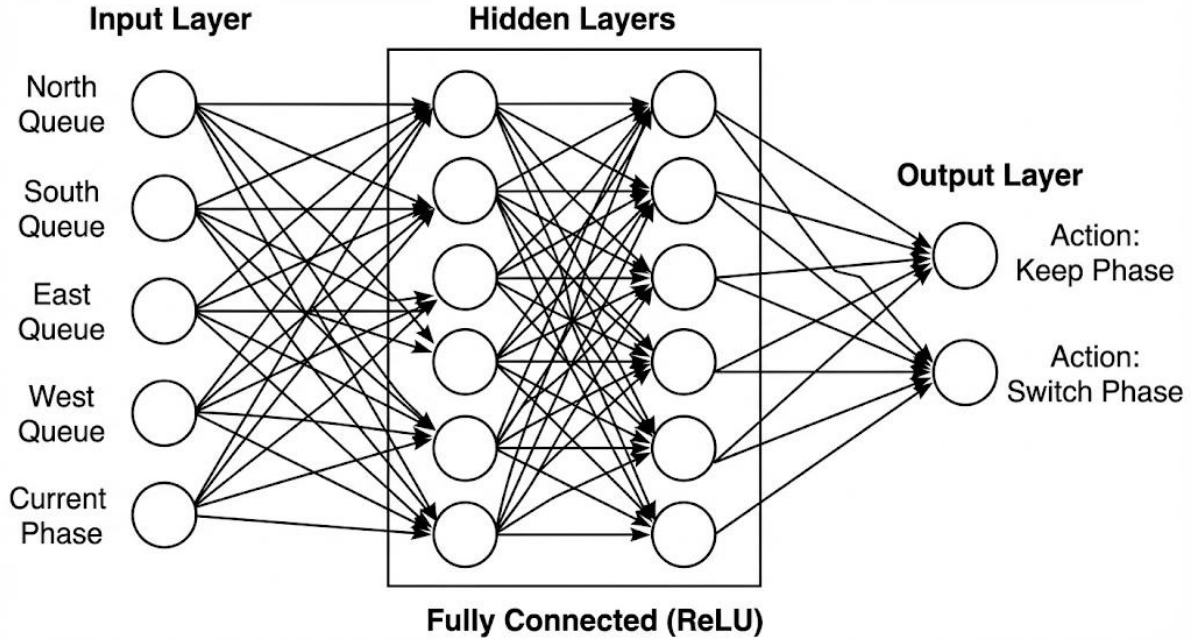


Figure 2: Deep Q-Network (DQN) Architecture – illustrating the Multi-Layer Perceptron (MLP) structure mapping traffic state inputs to optimal action Q-values.

4.3.4. Network Architecture Plan We plan to implement a Multi-Layer Perceptron (MLP) architecture:

- **Input Layer:** Will match the flattened size of the State Vector (S_t).
- **Hidden Layers:** We aim to use two fully connected layers with “ReLU” activation functions to capture complex traffic patterns.
- **Output Layer:** Will consist of two neurons representing the predicted Q-values for the possible actions ($Q(s, Keep)$, $Q(s, Switch)$).

4.4. Data Sources and Training Strategy

- **Computer Vision Data:** Our goal is Real-World Deployment. Therefore, we plan to utilize a YOLOv8 model pre-trained on real traffic datasets (e.g., COCO or BDD100K). To validate the system, we will test the model on real-world video streams from actual intersections. If the detection accuracy requires improvement due to environmental factors, we will perform Fine-Tuning using these real traffic samples to ensure robustness.
- **Reinforcement Learning Data:** The DQN agent is designed to learn via Online Interaction using self-generated synthetic data. We plan to implement a Replay

Buffer capable of storing approximately 50,000 transitions. The training process will target 1,000 episodes to ensure model convergence and policy optimization.

- **Simulation Configuration:** To ensure realism, traffic demand will be defined in “.rou.xml” files using Poisson distributions to mimic natural arrival rates. The planned vehicle mix consists of 85% private cars, 10% buses (for transit priority testing), and 5% emergency vehicles (for preemption validation).

4.5. Constraints & Challenges

- **Sim-to-Real Gap:** To address the stochastic nature of real-world drivers, we plan to introduce artificial noise into the simulation parameters (e.g., varying acceleration rates). This approach aims to enforce robust policy learning.
- **Occlusion:** To mitigate view blocking by large vehicles (e.g., a bus hiding a car), DeepSORT's Kalman Filter will be utilized to predict the positions of hidden objects. This will ensure the maintenance of accurate queue counts even when visual contact is lost.
- **Safety & Reliability:** Recognizing the 'black-box' risk of Neural Networks, we have designed a Rule-Based Safety Layer that will strictly enforce 'Minimum Green Time.' Additionally, a Fallback Mode is planned to revert the system to fixed-time schedules if the sensor confidence drops below a safety threshold.

4.6. Development Stages

The project execution followed a structured critical path:

1. **Environment Setup:** Implementation and calibration of the SUMO Simulation Environment and network topology.
2. **Agent Training:** Planned iterative training phase of the DQN agent using the replay buffer to achieve convergence.
3. **Stress Testing:** Future validation of the Priority Protocol using emergency scenarios against a fixed-time baseline.

5. Expected Achievements

5.1. Primary Objectives

The primary goal is to design an autonomous traffic light system that replaces static controllers with AI, focusing on three core pillars:

- **Real Time Optimization:** Creating a closed-loop system using a "Camera-as-a-Sensor" approach to minimize waiting times based on live visual data.
- **Prioritization of Critical Services:** Implementing a dynamic Priority Protocol to ensure immediate passage for emergency vehicles and public transit.

- **Accessibility and Cost Effectiveness:** Demonstrating a software-centric solution that upgrades intersections without expensive physical infrastructure changes.

5.2. Expected System Capabilities

By the end of the development phase, the system is expected to deliver the following functional modules:

- **Comprehensive Simulation Environment:** A fully functional SUMO Simulation Environment mirroring a complex intersection topology.
- **Advanced Perception Module:** A robust Computer Vision Module (YOLO/DeepSORT) capable of detecting, classifying, and tracking vehicles under varying densities.
- **Autonomous Control Logic:** An intelligent DQN Agent that autonomously manages traffic phases to maximize throughput.
- **Fail-Safe Safety Mechanisms:** A deterministic Safety Layer that enforces minimum green times and automatically reverts to a Fallback Mode upon detecting sensor failure.
- **Interactive User Interface:** A web-based Dashboard displaying real-time video feeds, traffic telemetry, and system status for operator monitoring.

6. Tools and Technologies

6.1. Computer Vision (YOLOv8)

YOLO (You Only Look Once) is a state-of-the-art, real-time object detection architecture based on Convolutional Neural Networks (CNN). Unlike traditional two-stage detectors that scan images multiple times, YOLO treats detection as a single regression problem, dividing the image into a grid to predict bounding boxes and probabilities simultaneously. **Why selected:** SUMO's TraCI (Traffic Control Interface) is the key enabler for this project. It will allow us to connect our Python-based DQN Agent to the simulation loop, enabling the retrieval of synthetic visual data and the execution of traffic light commands in a safe, risk-free environment.

6.2. Object Tracking (DeepSORT)

DeepSORT (Simple Online and Realtime Tracking with a Deep Association Metric) is a tracking algorithm that integrates motion prediction with visual appearance information. It utilizes a Kalman Filter to predict future object positions and a pre-trained CNN to generate appearance descriptors, allowing it to re-identify objects even after they have been temporarily occluded. **Why selected:** Standard detection (YOLO) has no "memory" of previous frames. DeepSORT allows us to bridge this gap, assigning unique IDs to vehicles to calculate "True Waiting Time" and track queue progression accurately.

6.3. Simulation & Development (SUMO / Python)

SUMO (Simulation of Urban MObility) is an open-source, microscopic traffic simulation package designed to model large road networks. It simulates the movement of each individual vehicle explicitly, considering physical parameters such as acceleration, deceleration, and lane-changing models. **Why selected:** SUMO's TraCI (Traffic Control Interface) is the key enabler for this project. It will allow us to connect our Python-based DQN Agent to the simulation loop, enabling the retrieval of synthetic visual data and the execution of traffic light commands in a safe, risk-free environment.

6.4. Web & Cloud Infrastructure

The system's backend and data management rely on a modern cloud-native stack:

- **FastAPI:** A high-performance web framework for building APIs with Python 3.7+, selected for its asynchronous support and automatic Swagger documentation generation.
- **PostgreSQL (Neon.tech):** A robust, open-source relational database selected to store historical traffic logs and system performance metrics. We plan to utilize a serverless implementation via Neon.tech for scalability.
- **Render:** A unified cloud platform that will be used to host and deploy the web application, ensuring 24/7 availability for the dashboard users.

6.5. Development Libraries & Hardware

To ensure high-performance execution, the system leverages specific hardware acceleration and libraries:

- **PyTorch:** The primary Deep Learning framework selected for constructing and training the DQN agent.
- **OpenCV:** Will be utilized for real-time image pre-processing and video stream manipulation before feeding frames into the YOLO model.
- **NVIDIA CUDA:** Required for GPU acceleration, ensuring the computer vision pipeline runs in real-time without processing lag.

7. Success Criteria

Given the "Simulation-First" strategy, the project's success is quantitatively evaluated within the SUMO Simulation Environment. The primary objective is to validate the algorithmic logic and safety protocols in a controlled digital twin before considering physical deployment. The system is measured against a standard Fixed-Time Controller baseline using the following benchmarks:

- **Traffic Flow Efficiency:** A reduction of at least 20% in Average Waiting Time per vehicle under heavy simulated traffic loads compared to the baseline.
- **Queue Management:** A measurable decrease in Maximum Queue Length to prevent lane overflow and spillback within the simulation grid.
- **Detection Accuracy** A Mean Average Precision (mAP) of >90% for standard vehicles and >95% for emergency vehicles on the validation dataset.
- **Real-Time Performance:** Maintaining a processing speed of >30 FPS to prove that the Computer Vision pipeline is capable of real-world deployment.
- **Emergency Response Efficiency:** Zero Delay (0s) for emergency vehicles in 100% of the simulated test cases (Signal Preemption).
- **System Stability:** Successful activation of Fallback Mode within less than one signal cycle upon detecting sensor failure or data anomalies.

8. Project Requirements

8.1. Functional Requirements (FR)

ID	Category	Requirement Description
FR-01	Input	The system shall ingest continuous video data.
FR-02	Detection	The system shall detect objects within the video space.
FR-03	Classification	The system shall classify each detected object into an assigned category.
FR-04	Tracking	The system shall assign a unique identifier and track objects over time.
FR-05	Metric Calculation	The system shall calculate the waiting time for each lane.
FR-06	Metric Calculation	The system shall calculate the queue length in each lane.
FR-07	Decision	The system shall make a decision regarding the next traffic light phase.
FR-08	Control	The system shall transmit execution commands to the traffic light controller.
FR-09	Emergency Identification	The system shall identify the presence of emergency vehicles in the intersection.
FR-10	Emergency Priority	The system shall perform lane prioritization for emergency vehicles.

ID	Category	Requirement Description
FR-11	Transit Priority	The system shall perform prioritization for public transport vehicles.
FR-12	Safety	The system shall validate the legality of the command before sending it for execution.
FR-13	Self-Monitoring	The system shall monitor the reliability of the incoming data.
FR-14	Failure Mode	The system shall activate a backup procedure upon detecting a reliability anomaly.

8.2. Non-Functional Requirements (NFR)

ID	Category	Requirement Description
NFR-01	Data Source	The video input shall be received from a simulator (SUMO) or an external camera.
NFR-02	Classification Categories	The distinct vehicle categories are: Private Car, Bus, Emergency Vehicle.
NFR-03	Detection Algorithm	Detection shall be performed using the YOLO architecture.
NFR-04	Tracking Algorithm	Tracking shall be performed using the DeepSORT algorithm.
NFR-05	Decision Logic	Decision making shall be performed by a Deep Q-Network (DQN) agent.
NFR-06	Emergency Priority Method	Priority shall be executed using the Preemption method (stopping opposing traffic and providing immediate green).
NFR-07	Transit Priority Method	Priority shall be executed using the TSP method (extending existing green time).
NFR-08	Safety Legality	The central safety constraint is Minimum Green Time (e.g., 10 seconds).
NFR-09	Failure Condition	The backup procedure shall be triggered when the Confidence Score drops below a defined threshold.
NFR-10	Backup Configuration	In failure mode, the system shall operate in a Fixed-Time configuration based on historical data.
NFR-11	Communication Protocol	Communication with the traffic controller shall be conducted via the TraCI protocol.
NFR-12	Max Latency	The total processing time shall not exceed 100 milliseconds.

ID	Category	Requirement Description
NFR-13	Processing Rate	The minimum processing rate shall be 45 Frames Per Second (FPS).
NFR-14	Required Accuracy	Detection accuracy (mAP) shall be at least 90% (general) and 95% (emergency).
NFR-15	Required Efficiency	An improvement of 20% in waiting time and 15% in queue length (relative to a standard controller).
NFR-16	Emergency Time	The allowable delay time for an emergency vehicle is 0 seconds.
NFR-17	Hardware	Use of a GPU supporting CUDA is required.
NFR-18	Development Language	The system shall be written in Python.

9. Verification and Evaluation

9.1. Test Plan Overview

This chapter details the testing methodology that will be used to validate the system's performance. The evaluation will follow a rigorous A/B Testing approach, comparing the AI-driven solution against standard traffic control methods under identical simulation conditions.

9.2. Defined Test Scenarios

The system will be evaluated under diverse traffic scenarios to measure adaptability:

- **Light Flow:** Low traffic density.
- **Rush Hour (Heavy Load):** High-density traffic (saturation).
- **Emergency Events:** Planned validation of Signal Preemption logic.
- **Sensor Failure Mode:** Simulating camera blindness (occlusion/weather).

9.3. Test Environment

The testbed will utilize the SUMO platform, Python/TraCI interface, and a GPU workstation. The system will be tested on a complex intersection topology mirroring real-world constraints.

9.4. Evaluation Methodology:

To quantitatively measure performance, we plan to employ a scientific A/B Testing approach using a Baseline Comparison method:

- **Defining the Baseline:** A standard Fixed-Time Controller (e.g., 90-second static cycle) will serve as the control group.

- **Controlled Experiments:** Identical traffic scenarios using fixed random seeds will be executed for both the Baseline and the AI-Driven System to ensure scientific consistency.
- **Performance Calculation:** Improvement will be calculated by analyzing SUMO log files (tripinfo output) for key metrics such as Average Waiting Time and Queue Length.

9.5. Key Performance Metrics

- **Average Waiting Time (AWT):** The mean time (in seconds) a vehicle spends standing (velocity < 0.1 m/s).
- **Queue Length:** The maximum number of vehicles standing in a lane.
- **Throughput:** Total vehicles completing the route.

9.6. Verification Test Cases

The following table outlines the planned verification tests to be executed:

Case	Test Case	Expected Result
1	YOLO Vehicle Detection & Classification Feed raw frames from the simulation to the YOLO model to verify object detection.	The system is expected to detect all vehicles within the frame and correctly classify them (distinguishing "Bus" or "Emergency" from "Car").
2	DeepSORT Object Tracking Track a specific vehicle moving across the intersection for 50 consecutive frames, including moments of occlusion (hidden behind another vehicle).	The system should maintain the same unique Vehicle_ID for the vehicle throughout its trajectory, even after temporary occlusion.
3	Traffic Light Control via TraCI Send a specific "Switch Phase" command from the Python script to the SUMO simulator.	The traffic light in the simulation must change immediately according to the command, and virtual vehicles should react to the signal change.
4	Emergency Priority Protocol Inject an emergency vehicle (Ambulance) into one of the approaching lanes in the simulation.	The system must identify the "Emergency" class, terminate conflicting phases, and provide a green light within less than 3 seconds.
5	Wait Time Calculation Compare the system's calculated "Cumulative Waiting Time" against the ground truth logs provided by SUMO.	The discrepancy between the vision-based calculation and the simulator's true data is expected to be less than 1 second per vehicle.

Case	Test Case	Expected Result
6	Fallback Mode Trigger Simulate a camera failure by artificially dropping the vision module's confidence score to 0.	The system must detect the anomaly, disable AI control, and revert to a Fixed-Time schedule without freezing the intersection.
7	Safety Layer Constraints Force the DQN agent to request a phase change before the "Minimum Green Time" has elapsed.	The Rule-Based Safety Layer must override the request and hold the current phase until the minimum safety time has passed.

9.7. Evaluation Metrics

These benchmarks will be used to compare performance against the Fixed-Time baseline:

Case	Metric	Expected Result
1	Average Waiting Time Reduction Run a 1-hour "Rush Hour" simulation with the DQN Agent and compare it to a Fixed-Time controller.	The DQN-based system aims to achieve a reduction of at least 20% in average waiting time compared to the baseline.
2	Queue Length Management Measure the maximum queue length (in meters) accumulated on all lanes during the simulation.	The system is expected to reduce the maximum queue length by at least 15%, preventing spillback.
3	Emergency Vehicle Delay Measure the delay time experienced by emergency vehicles while crossing the intersection.	The delay should be reduced to 0 seconds in 100% of test cases due to the preemption protocol.
4	Throughput (Traffic Flow) Count the total number of vehicles that successfully exit the intersection within a 1-hour timeframe.	The system should achieve higher throughput compared to standard controllers under heavy traffic.
5	Detection Accuracy (mAP) Evaluate the YOLO model's performance against ground truth data.	The model is targeted to achieve a Mean Average Precision (mAP@0.5) of >90% for standard vehicles and >95% for emergency vehicles on the validation dataset.
6	System Latency Measure the time elapsed from frame capture to signal command execution.	Total processing latency must remain under 100 milliseconds to ensure real-time responsiveness.

10. User Interface Design

10.1. Overview & Layout

The system is designed to feature a Web-Based Dashboard for "At-a-Glance" monitoring. It will utilize a Quadrant Grid Layout, dividing the screen into four dedicated zones (North, South, East, West). This arrangement aims to mirror the physical intersection geometry, allowing operators to view all directions simultaneously without the need for scrolling.

10.2. UI Components

Each directional quadrant will consist of two functional layers:

- **Live Video Feed:** A high-contrast display streaming real-time footage (via RTSP), overlaid with Camera ID and timestamps.

Telemetry Data: A status panel displaying real-time metrics derived from the AI processing unit:

- **Light Status:** Current state indicator (GREEN / RED).
- **Wait Time:** Counter in seconds representing lane delay.
- **Vehicle Queue:** Real-time count of detected vehicles.

10.3. System Behavior

- **Real-Time Synchronization:** The interface auto-refreshes every 2.0 seconds to fetch the latest DQN model decisions.
- **Logic Visualization:** The UI visually reinforces safety constraints, ensuring that North/South axes and East/West axes always switch states in unison.
- **Operational Mode:** The dashboard operates in Read-Only mode to prevent accidental interference during live monitoring.

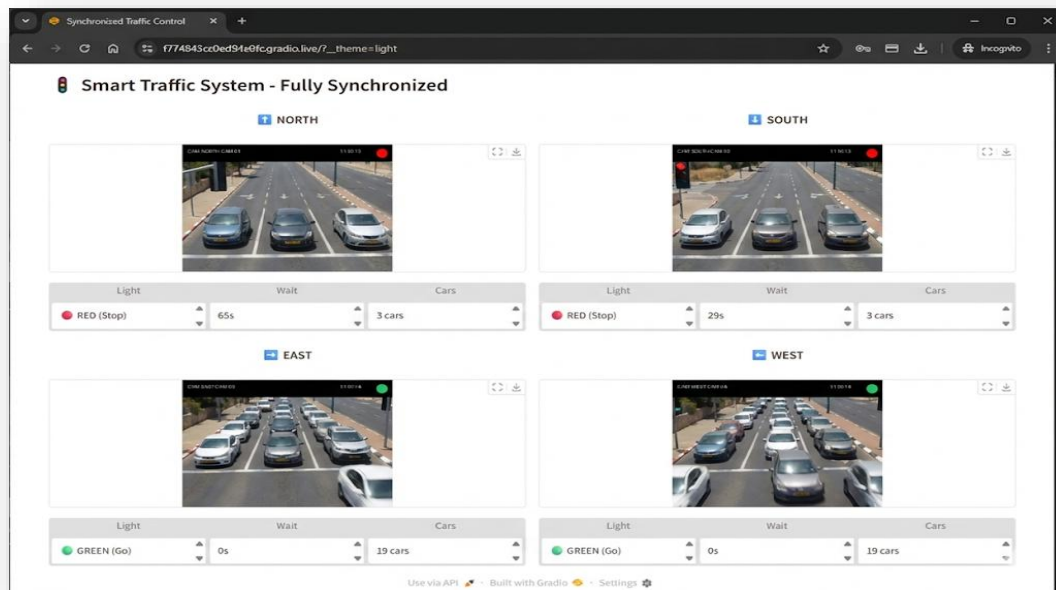


Figure 4: Smart Traffic System Dashboard (UI) – displaying live video feeds, real-time vehicle counts, and traffic light status

11. Use Case Diagram Description

11.1. Overview

This diagram defines the functional requirements and authorized operations for the Smart Traffic Management System, distinguishing between standard operators and system administrators.

11.2. Actors

- User (Base Actor): A standard operator responsible for daily monitoring and basic control.
- Admin (Specialized Actor): A manager who inherits all User capabilities (via generalization) and gains exclusive access to system configuration and user management.

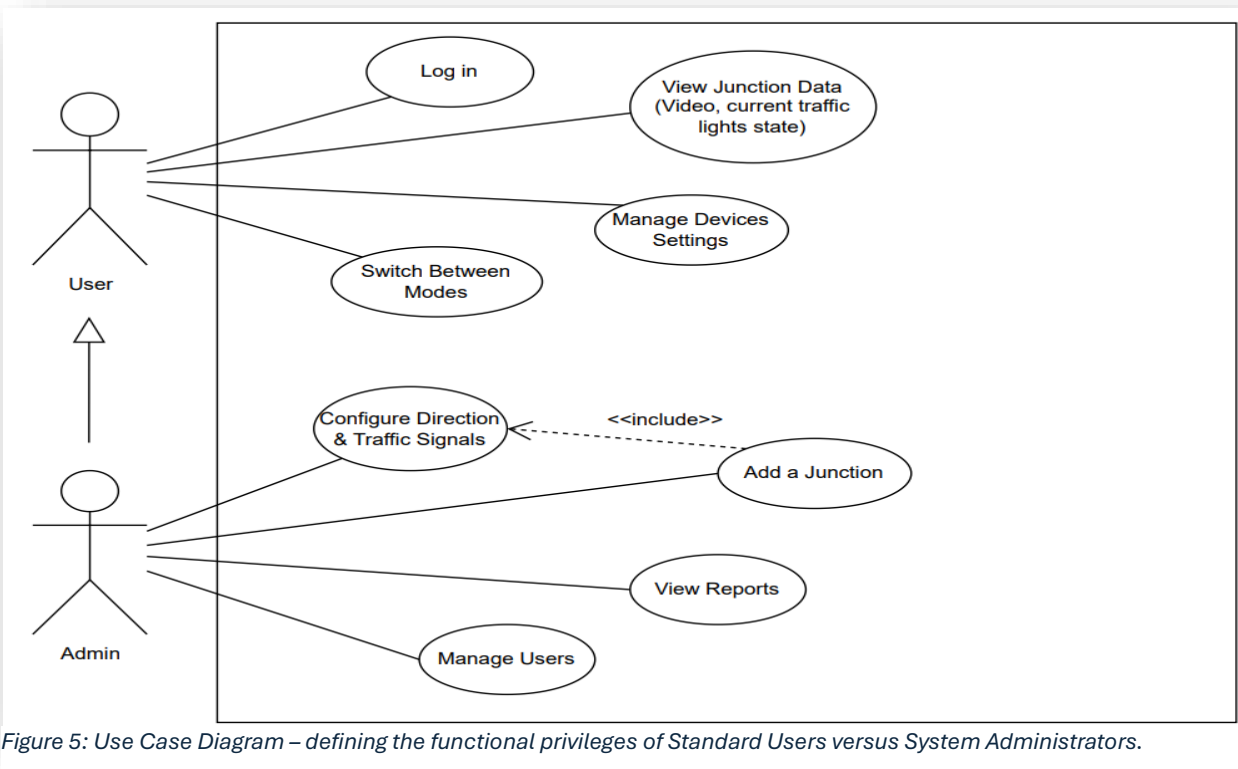


Figure 5: Use Case Diagram – defining the functional privileges of Standard Users versus System Administrators.

11.3. Functional Capabilities

The system supports two levels of access:

- **Common Features (User & Admin):**
 - Log in: Secure system entry.
 - View Junction Data: Monitoring live video (RTSP) and traffic light status.
 - Manage Device Settings: Basic configuration of edge devices.
 - Switch Between Modes: Toggling between AI control and manual override.

- **Admin-Only Features:**
 - Manage Users: Creating and removing operators.
 - View Reports: Accessing performance analytics and logs.
 - Add a Junction: Registering new intersections.

11.4. Dependencies The diagram utilizes an <<include>> relationship:

The "Add a Junction" use case mandatorily includes "Configure Direction & Traffic Signals", ensuring no junction is created without proper signal configuration.

12. Activity Diagram Description –

12.1. Overview

The Activity Diagram (presented as a Cross-Functional Flowchart) visualizes the real-time operational loop of the system. It details the sequential flow of control from data acquisition to signal execution, categorized by the component responsible for each step.

12.2. Structure (Swimlanes)

The diagram is divided into three horizontal swimlanes, representing the physical and logical layers of the system:

- **Environment / Sensors:** Handles raw data acquisition (Video Frames) from the field.
- **Processing Unit:** The core logic layer where image processing (YOLO/DeepSORT), AI decision-making (DQN), and safety validation occur.
- **Traffic Controller:** The physical hardware interface responsible for executing the final light signal updates.

12.3. Process Flow

The logic follows a continuous control loop divided into three operational phases:

- **Phase 1: Input & Validation** The cycle begins with capturing a video frame. The system immediately validates the input quality.
 - If Invalid: The system enters Fallback Mode, executing a safe, pre-defined Fixed-Time Cycle.
 - If Valid: The system proceeds to detect and track vehicles using computer vision algorithms to calculate real-time traffic metrics.
- **Phase 2: Decision Making & Priority** Before optimizing traffic, the system checks for Emergency Vehicles.
 - Emergency Detected: A priority protocol is triggered immediately to clear the path.

- Standard Operation: If no emergency is found, the DQN Agent analyzes the state representation and selects the optimal action (Switch or Hold phase).
- **Phase 3:** Constraints & Execution The selected action is validated against safety constraints (e.g., Minimum Green Time).
 - Execution: Based on the validation, the system decides whether to Hold the current phase or Switch lights.
 - Output: The command is sent to the Traffic Controller to update the physical signals, and the loop restarts.

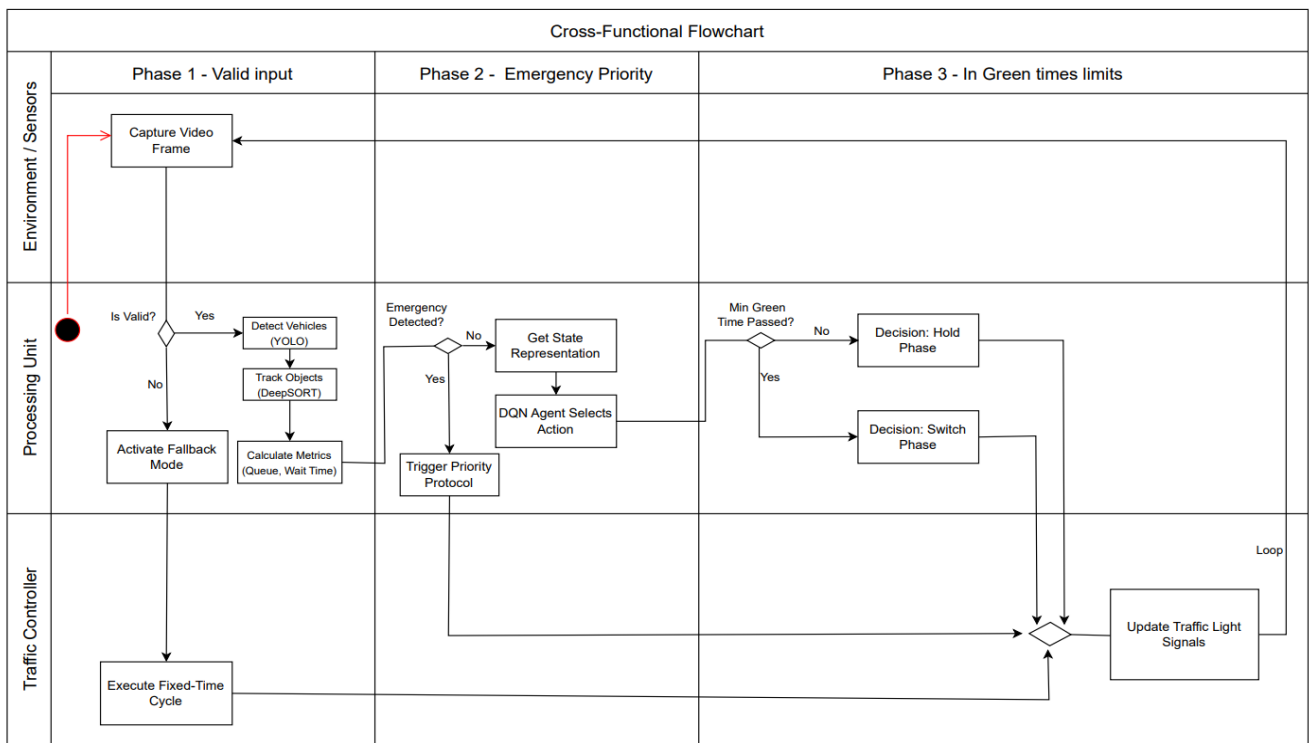


Figure 6: Activity Diagram (Cross-Functional Flowchart) – illustrating the real-time control loop from sensor input to signal execution.

13. Deployment Diagram Description –

13.1. Overview

The Deployment Diagram illustrates the physical architecture of the system, mapping software artifacts to their execution environments. The architecture follows a Hybrid Cloud-Edge model, ensuring that critical real-time processing occurs locally at the intersection ("Edge"), while data aggregation and management are handled centrally in the Cloud.

13.2. Nodes & Execution Environments

The system is distributed across three distinct zones:

- **Cloud Infrastructure (PaaS - Platform as a Service):** The central backend environment hosting the system's management layer.
 - **Backend API Server:** Hosted on Render, running the Python/FastAPI application.
 - **Central Database:** A managed PostgreSQL instance hosted on Neon.tech, ensuring scalable data storage.
- **Smart Intersection Environment (Edge):** The physical location where traffic control takes place.
 - **Edge Computer:** An industrial PC acting as the system's "Brain." It hosts the Control Logic (Python) and the Trained DQN Model artifact.
 - **Field Devices:** Includes the IP Camera (connected via PoE) and the Traffic Light Controller (PLC).
- **User Workstation:** The client-side device (PC/Mobile) running a standard Web Browser to access the dashboard.

13.3. Communication Protocols

The components communicate using specific protocols optimized for their tasks:

- **RTSP (Real-Time Streaming Protocol):** Transfers high-speed video feed from the IP Camera to the Edge Computer for processing.
- **GPIO (General Purpose I/O):** A low-latency physical connection sending binary commands from the Control Logic to the Traffic Controller (PLC).
- **HTTPS (API Sync):** The synchronization link between the Edge and the Cloud. The Edge Computer uses this secure channel to Push statistical data and Pull configuration updates.
- **HTTPS / WebSocket:** Connects the User's browser to the Cloud Server for real-time dashboard viewing.

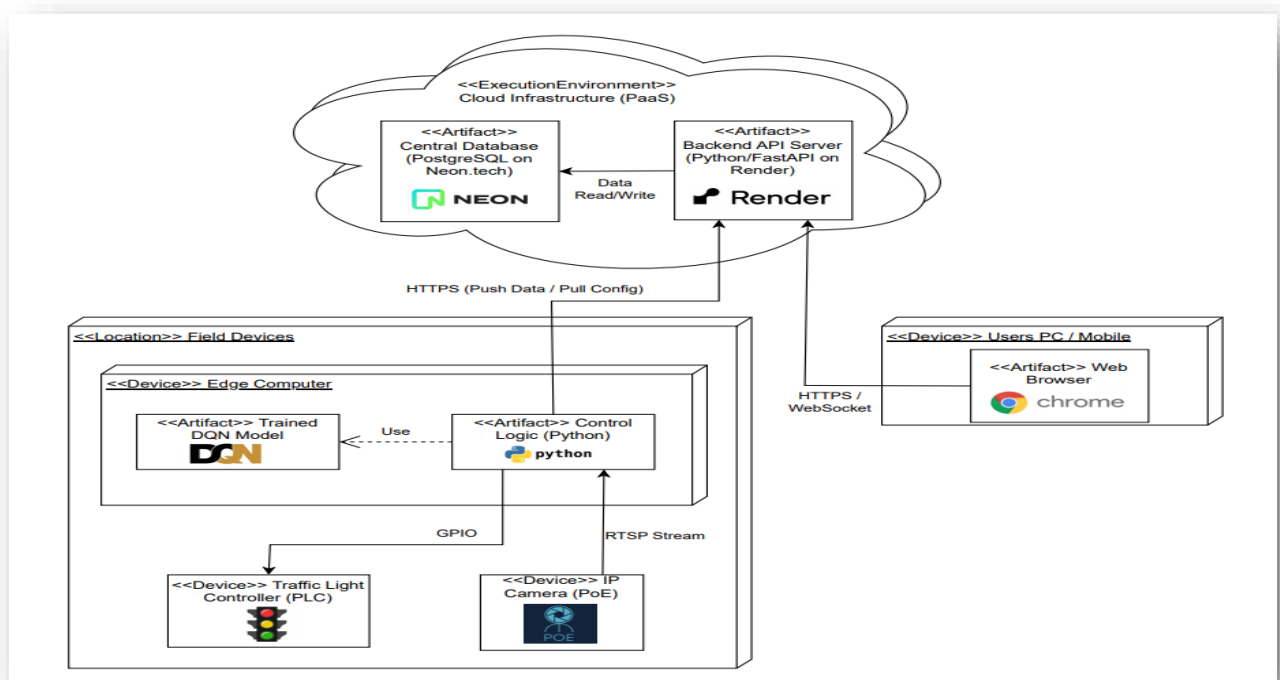


Figure 7: Deployment Diagram – showing the physical architecture, hardware components, and communication protocols (Cloud-Edge Hybrid Model).

14. References

- Akçelik, R. (1994). Gap-acceptance modeling by traffic signal logic. *Transportation Research Record*, 1457, 6-16. Retrieved from <https://onlinepubs.trb.org/Onlinepubs/trr/1994/1457/1457-008.pdf>
- Akçelik, R. (2010). Fundamental traffic variables in adaptive control and the SCATS DS parameter. *Proceedings of the 24th ARRB Conference*, Melbourne, Australia. Retrieved from https://www.researchgate.net/publication/265339269_Fundamental_Traffic_Variables_in_Adaptive_Control_and_the_SCATS_DS_Parameter
- Department of Transport (UK). (1995). *SCOOT: A traffic responsive method of coordinating signals* (Traffic Advisory Leaflet 4/95). London, UK. Retrieved from <https://tsrgd.co.uk/pdf/tal/1995/tal-4-95>
- Dresner, K., & Stone, P. (2008). A multiagent approach to autonomous intersection management. *Journal of Artificial Intelligence Research*, 31, 591-656. Retrieved from <https://www.cs.utexas.edu/~aim/papers/JAIR08-dresner.pdf>
- Koonce, P., Rodegerdts, L., Lee, K., Quayle, S., Beaird, S., Braud, C., ... & Tarnoff, P. (2008). *Traffic signal timing manual* (Report No. FHWA-HOP-08-024). Federal Highway Administration. Retrieved from https://nacto.org/wp-content/uploads/2-7_FHWA-Traffic-Signal-Timing-Manual-ch-6_2008.pdf
- Liang, X., Du, X., Wang, G., & Han, Z. (2018). Deep reinforcement learning for traffic light control in vehicular networks. *arXiv preprint arXiv:1803.11115*. Retrieved from <https://arxiv.org/pdf/1803.11115>
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., ... & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529-533. Retrieved from <https://storage.googleapis.com/deepmind-media/dqn/DQNNaturePaper.pdf>
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 779-788). Retrieved from <https://arxiv.org/pdf/1506.02640>
- Wang, Y., Li, Z., & Zhang, Y. (2024). Traffic co-simulation framework empowered by infrastructure camera sensing. *arXiv preprint arXiv:2412.03925*. Retrieved from <https://arxiv.org/pdf/2412.03925>
- Wojke, N., Bewley, A., & Paulus, D. (2017). Simple online and realtime tracking with a deep association metric. In *2017 IEEE International Conference on Image Processing (ICIP)* (pp. 3645-3649). IEEE. Retrieved from <https://arxiv.org/pdf/1703.07402>

15. Appendices

15.1. Appendix A: State Space Representation Pipeline

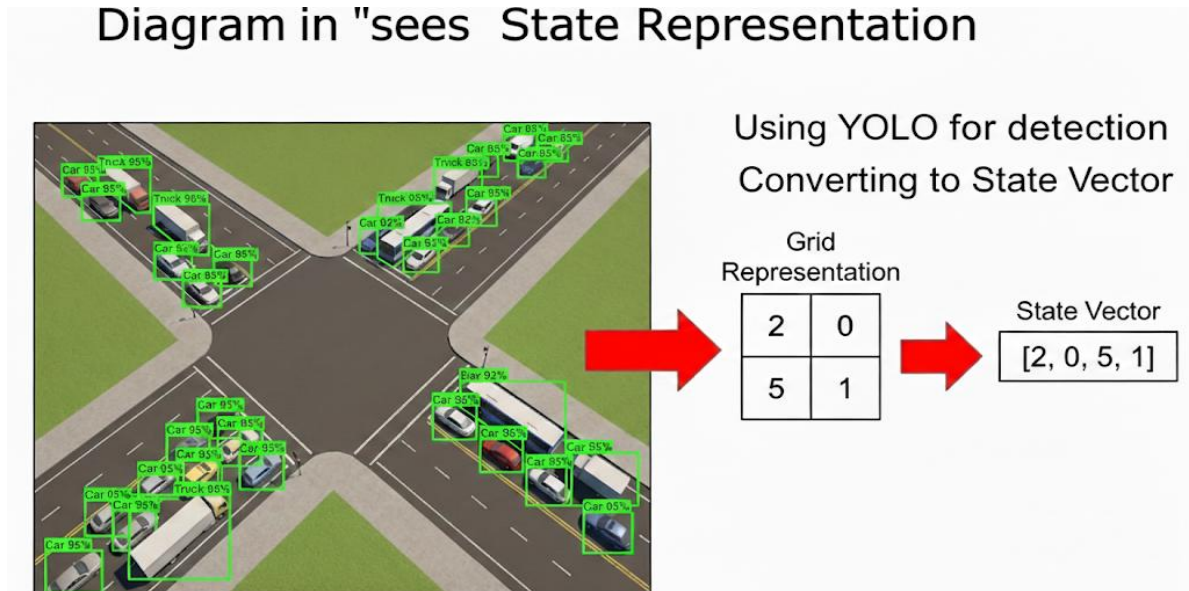


Figure 3: State Space Transformation Pipeline – demonstrating the conversion of raw visual data (YOLO detections) into a structured numerical tensor for the DQN agent.

15.2. Appendix B: Use of Generative AI

15.2.1. AI Tools Utilized

The following AI tools were used to assist in literature review, code debugging, and technical documentation drafting.

Tool Name	Purpose	URL
Gemini (Google)	Core assistant for literature summary, critical analysis, code generation (Python/TraCI), and documentation writing.	https://gemini.google.com/
Google NotebookLM	Analyzing specific academic papers and synthesizing research gaps.	https://notebooklm.google.com/

15.2.2. Prompts

A. Literature Review & Concept Clarification

- "We are analyzing existing traffic control methods. What are the known functional limitations of Inductive Loop Detectors compared to Computer Vision sensors, specifically regarding queue length estimation?"

- *"I have pasted the 'Conclusion' section from a research paper regarding V2X implementation costs below. Please summarize the key barriers to entry mentioned in the text."*
- *"Explain the technical difference between SCATS and SCOOT systems, focusing on their optimization algorithms."*

B. Critical Analysis & Gap Identification

- *"Play the role of a skeptic engineer: What are the main environmental challenges (e.g., weather, lighting) that could cause a Computer Vision traffic system to fail?"*
- *"We want to ensure our Reinforcement Learning agent does not make dangerous decisions. Suggest a logic flow for a 'Rule-Based Safety Layer' that overrides the AI if Minimum Green Time is violated."*

C. Technical Implementation & Algorithms

- *"We are using DeepSORT for object tracking. Explain how the Kalman Filter helps predict the position of a vehicle when it is temporarily occluded by a truck."*
- *"Write a Python function using the TraCI library to check if an emergency vehicle is present in the current lane and trigger a phase switch."*
- *"Help me format these functional requirements into a standard IEEE engineering table format."*